

File System

Antonio Pecchia

antonio.pecchia@unisannio.it

Dipartimento di Ingegneria
Laurea in Ingegneria Informatica
Sistemi Operativi



UNIVERSITÀ DEGLI STUDI
DEL SANNIO Benevento

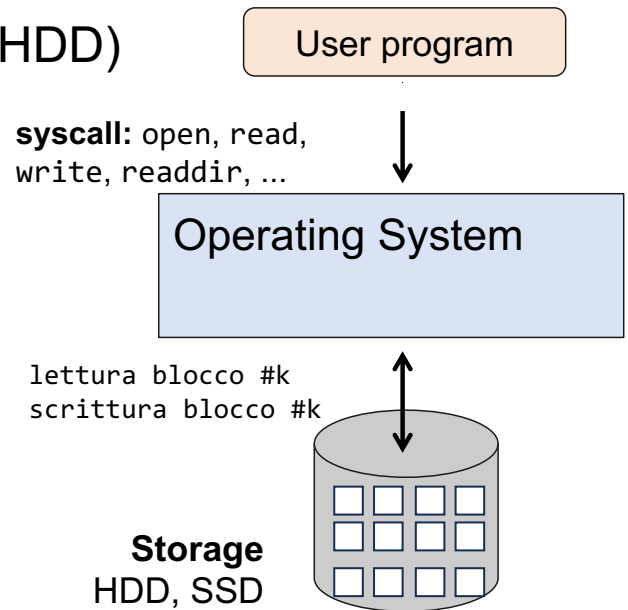
Salvataggio informazioni a lungo termine

- Requisiti per il salvataggio delle informazioni a **lungo termine**:
 - possibilità di salvare enormi quantità di informazioni;
 - le informazioni devono permanere oltre la fine del processo che le usa;
 - le informazioni devono poter essere acquisite, eventualmente in contemporanea, da più processi.

La tecnologia

- Tipicamente **dischi magnetici** (HDD) e **unità a stato solido** (SSD)*:

- *HDD e SSD saranno trattati nell'ambito dell'**input/output**;*
- al momento, assumiamo che supportino un'interfaccia del tipo "leggi/scrivi blocco".



* Hard Disk Drive (HDD)
Solid State Drive (SSD)

File

- **File**: unità logica di informazioni, correlate e registrate nella memoria secondaria, cui è stato assegnato un identificativo (nome), persistente, creato da un processo:
 - sono un **meccanismo di astrazione** per salvare informazioni sul disco e leggerle in seguito;
 - un file Linux è una **sequenza di byte** contenenti informazioni arbitrarie.

File system

- **File system**: parte del sistema operativo responsabile per la gestione e l'organizzazione dei **file**;
- Alcuni **compiti** di un file system:
 - implementazione e persistenza dei file;
 - recupero-accesso dei file;
 - un meccanismo di naming;
 - condivisione e la protezione dei file;
 - ...

Esempi: FAT12/FAT16 (MS-DOS), NTFS
(Windows), ext2/ext4 (Linux), APFS (macOS/iOS);

FILE E DIRECTORY (VISTA "UTENTE")

Cap 4: par. 4.1, 4.2

File naming

- Quando un processo crea un file, gli attribuisce un **nome**:
 - lunghezza: **FAT12**: 8 characters (esteso a 255);
Ext4: 255 characters.
 - **estensione**: in alcuni sistemi (tipo UNIX), le estensioni sono solo *convenzioni*, non forzate dal SO.

Estensione	Significato
.bak	File di backup
.c	Programma sorgente in linguaggio C
.gif	Immagine in Compuserve Graphical Interchange Format
.html	Documento HTML (world wide web hypertext markup language)
.jpg	Immagine codificata con lo standard JPEG
.mp3	Musica codificata in formato audio MPEG layer 3
.mpg	Filmato codificato in formato audio MPEG standard
.o	File oggetto (output da compilatore, non ancora linkato)
.pdf	Documento in formato Adobe PDF (portable document format)
.ps	File PostScript
.tex	Input per il programma di formattazione TEX
.txt	File di testo generico
.zip	Archivio compresso

Struttura di un file

Caso (a): sequenza di byte. Il "significato" è *imposto* dai programmi utente. I SO tradizionali, tipicamente, adottano questo approccio.

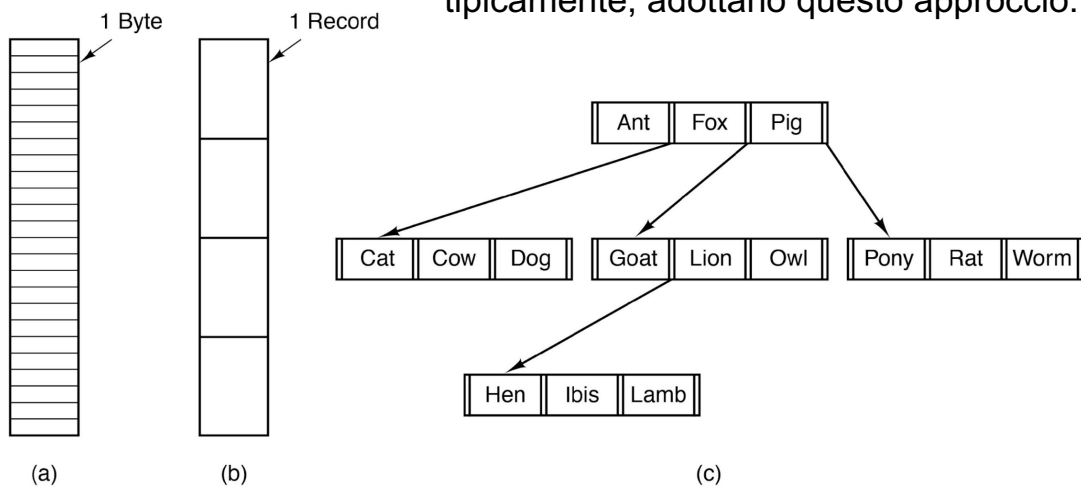


Figure 4-2. Struttura dei file. (a) Byte sequence. (b) Record sequence. (c) Tree.

Tipi di file

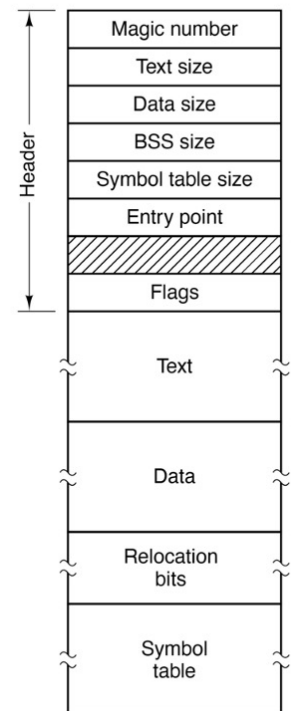
- **File "normali":** sono sequenze di byte che possono contenere informazioni qualsiasi (dati, programmi sorgente, programmi oggetto):
 - file **ASCII** (righe di testo) o **binari**.
- **Directory:** file di sistema usati per mantenere la struttura del file system;
- **File speciali**, a **caratteri** per astrarre i dispositivi seriali di I/O (terminali, reti) o a **blocchi** (dischi).

Tipi di file (esempio)

- L'utilità **file** può essere usata per esaminare il tipo di file.

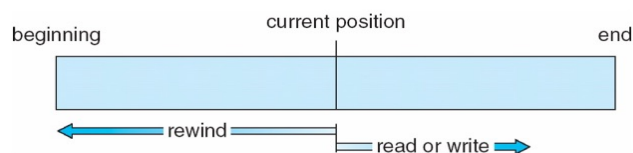
```
$ file a.out
a.out: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=6ffcfea2d8c91666dbc5a10922d7e83c79fa8049, for
GNU/Linux 3.2.0, not stripped
```

**File
eseguibile**



Accesso ai file

- **Sequenziale**: un processo legge i byte (o record) in ordine; non può né saltare, né leggere in ordine sparso.



- **Casuale**: i byte (record) possono essere letti in qualsiasi ordine.

Attributi dei file

- ❑ Gli attributi (detti anche **metadati**) sono informazioni associate ai file;
- ❑ Esempi di attributi:

Attributo	Significato
Protezione	Chi può accedere al file e in che modalità
Password	Password necessaria per accedere al file
Creatore	ID della persona che ha creato il file
Proprietario	Proprietario attuale
Flag di sola lettura	0 per lettura/scrittura; 1 per sola lettura
Flag di file nascosto	0 per normale; 1 per non visualizzare negli elenchi
Flag di file di sistema	0 per file normali; 1 per file di sistema
Flag di file archivio	0 per già sottoposto a backup; 1 per file di cui fare il backup
Flag ASCII/binario	0 per file ASCII; 1 per file binari
Flag di accesso casuale	0 per accesso sequenziale; 1 per accesso casuale
Flag temporaneo	0 per normale; 1 per cancellare il file al termine del processo
Flag di file bloccato	0 per non bloccato; non zero per bloccato
Lunghezza del record	Numero di byte nel record
Posizione della chiave	Offset della chiave in ciascun record
Lunghezza della chiave	Numero di byte del campo chiave
Data e ora di creazione del file	Data e ora di quando il file è stato creato
Data e ora di ultimo accesso al file	Data e ora di quando è avvenuto l'ultimo accesso al file
Data e ora di ultima modifica al file	Data e ora di quando è avvenuta l'ultima modifica al file
Dimensione attuale	Numero di byte nel file
Dimensione massima	Numero di byte di cui può aumentare il file

Operazioni su file ed esempi*

- ❑ Alcune operazioni sui file:
 - create, delete, open, read, write, append, seek, ...

```
int fd = open("foo.txt", O_RDONLY);
char buf[512];
ssize_t bytes_read = read(fd, buf, 512);
close(fd);
printf("read %zd: %s\n", bytes_read, buf);
```

opening a file returns a handle (file descriptor) for future operations.

*Sezioni 4.1.6 e 4.1.7 da consultare a fini di ripetizione.

Path (percorsi) assoluti e relativi

- ❑ Percorso **assoluto**: indica come giungere al file/directory partendo dalla directory radice (**root**);
- ❑ Percorso **relativo**: fa riferimento alla *directory corrente* (o *directory di lavoro*);
- ❑ Ogni directory ha due *voci speciali*: **.** (*current directory*) e **..** (*parent directory*).

```
studente@opsys:~$ pwd
/home/studente
studente@opsys:~$ mkdir mytest
studente@opsys:~$ touch mytest/myfile.txt
studente@opsys:~$ ls /home/studente/mytest/myfile.txt
/home/studente/mytest/myfile.txt
```

Operazioni su directory

- ❑ Alcune operazioni sulle directory:
 - create, delete, opendir, readdir, rename, ...

```
DIR *dirp = opendir(".");

struct dirent *dirent;
while ((dirent = readdir(dirp)))
    printf("%s\n", dirent->d_name);

closedir(dirp);
```

*Sezione 4.2.4 da consultare a fini di ripetizione.

FILE E DIRECTORY

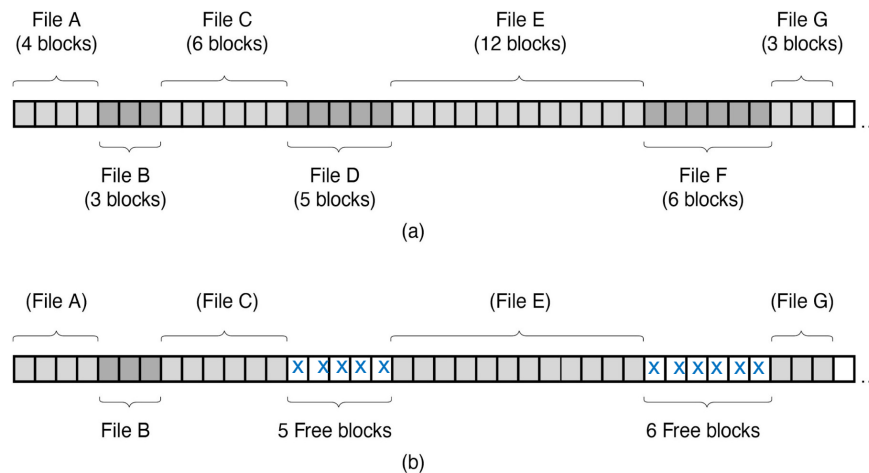
Cap 4: par. 4.3.2 e 4.3.3

Implementazione dei file

- **Modalità** con cui i blocchi del disco sono assegnati ai file, e **come** "si tiene traccia" di quali blocchi sono associati ad un determinato file:
 - allocazione contigua;
 - allocazione a liste concatenate;
 - allocazione a liste concatenate *con tabella in memoria*;
 - i-node.

Allocazione contigua

- Un file è memorizzato come una sequenza contigua di blocchi.



Allocazione contigua

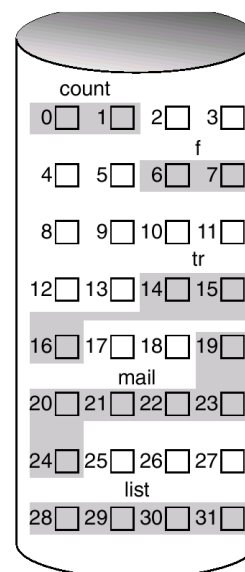
Vantaggi:

Per reperire tutto il file sono necessarie solo la posizione di inizio (block #) e la lunghezza (numero di blocchi).

Svantaggi:

Frammentazione esterna;

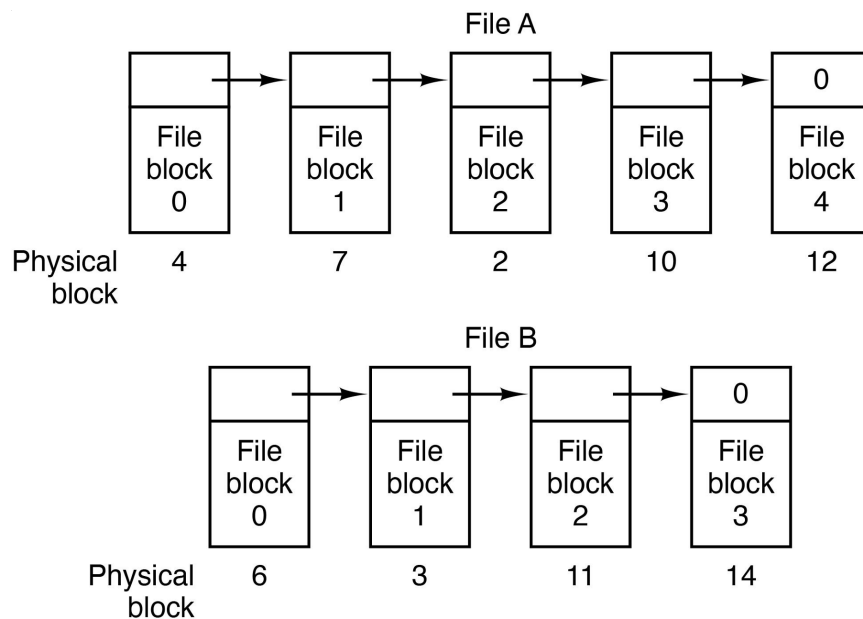
I file **non** possono crescere in dimensione.



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Allocazione a liste concatenate



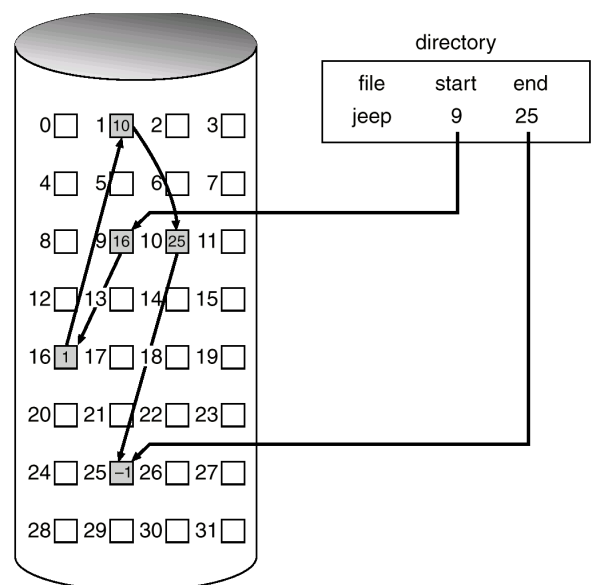
Allocazione a liste concatenate

Vantaggi:

Non c'è frammentazione esterna;
Minor costo di allocazione.

Svantaggi:

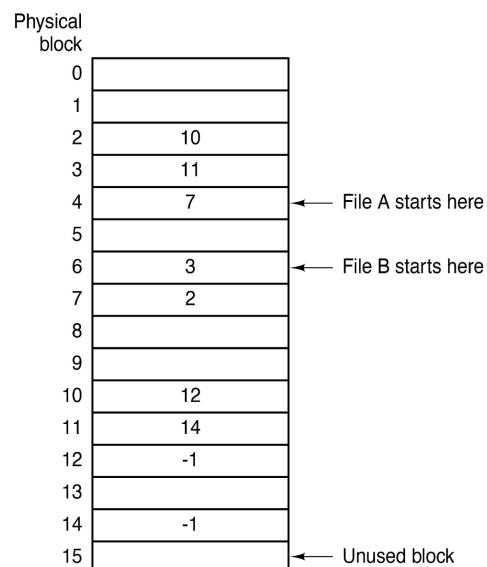
Possibilità di errore se un link viene danneggiato;
Overhead di spazio (ossia lo spazio occupato dai puntatori);
Costo della ricerca di un blocco.



Liste concatenate con tabella in memoria

Il puntatore al blocco è messo in una **tabella in memoria**, detta **File Allocation Table (FAT)**. Per es., file system originale di MS-DOS.

Svantaggi: dimensione della tabella (per dischi di grosse dimensioni); occupazione di memoria.

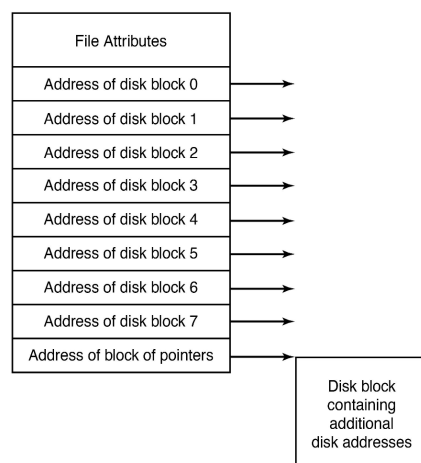


i-node (*index node* o *inode*)

- Struttura dati che elenca gli attributi e gli indirizzi dei blocchi del file (UNIX, idea simile in Windows NTFS).

L'**i-node** deve essere in memoria quando il file è aperto;

Detto **k** il numero massimo di file che possono essere aperti contemporaneamente ed **n** la dimensione dell'i-node, l'occupazione degli i-node sarà **k x n**;



UNIX i-node

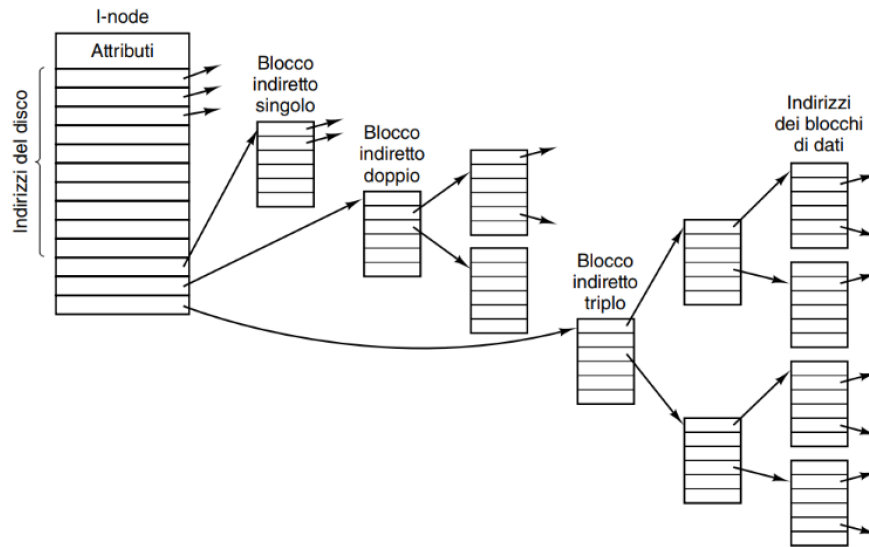


Figura 4.35 Un i-node UNIX.

Implementazione delle directory

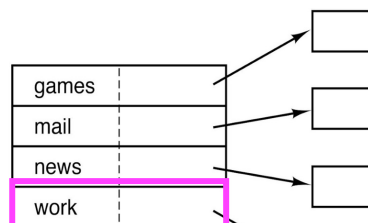
- La funzione principale del sistema delle directory è di mappare il nome del file sulle informazioni necessarie per localizzarne i dati sul disco*.

voce (entry)
della directory

games	attributes
mail	attributes
news	attributes
work	attributes

(a)

voce (entry)
della directory



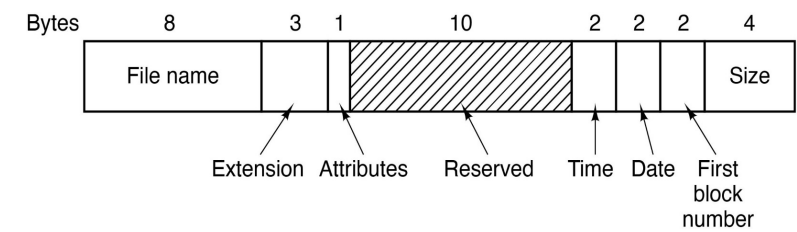
(b)

Data structure
containing the
attributes

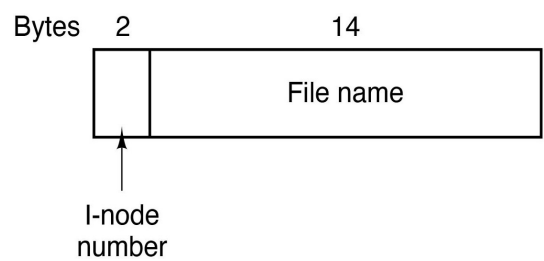
*NOTA: per il momento, non è stato ancora introdotto il meccanismo con cui, dato il percorso di un file, si localizza la corrispondente voce di directory sul disco.

Esempi di directory entry

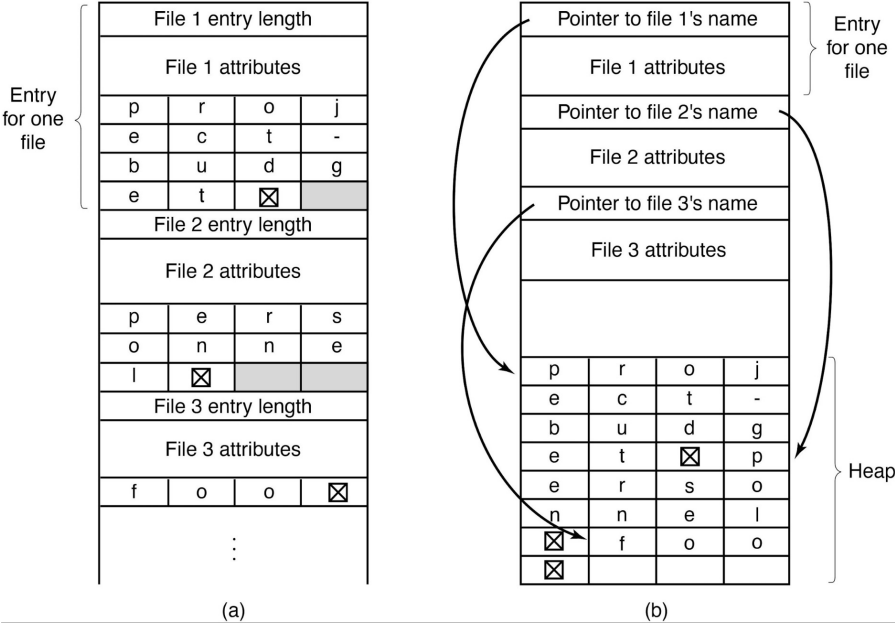
□ FAT12/16 - metodo (a):



□ UNIX - metodo (b), con i-node:



Gestione file name "lunghi" e a dimensione variabile



Comandi trattati durante la lezione

- ❑ **file**: determina il tipo del file;
- ❑ **ls** (opzione **-i**): mostra l'**index number** del file;
- ❑ **wc** (opzione **-c**): conteggio dei byte;
- ❑ **du** (opzioni **-sh**): disk usage.

```
$ file mutex.c  
mutex.c: C source, ASCII text
```

```
$ ls -li mutex.c  
925612 -rw-r--r-- 1 studente studente 2290 nov 13 15:01 mutex.c
```

```
$ wc -c mutex.c  
2290 mutex.c
```

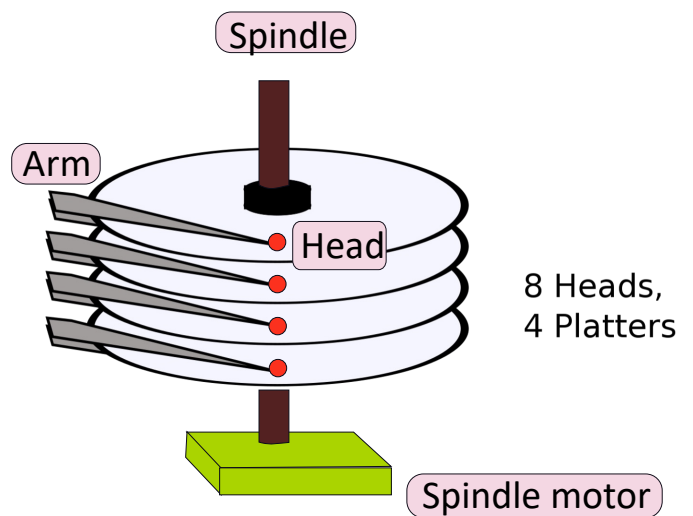
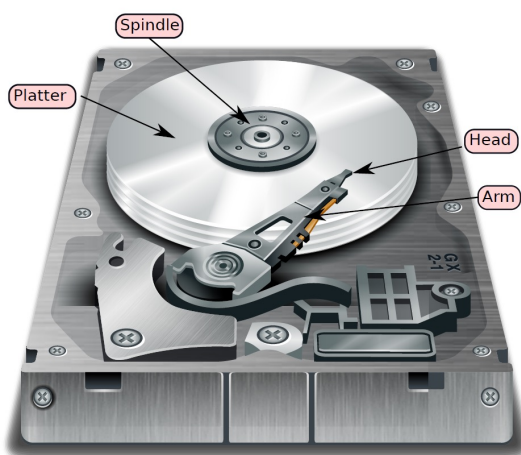
```
$ du -sh mutex.c  
4,0K mutex.c
```


DISCHI MAGNETICI (HDD)

Cap 5: par. 5.4.1

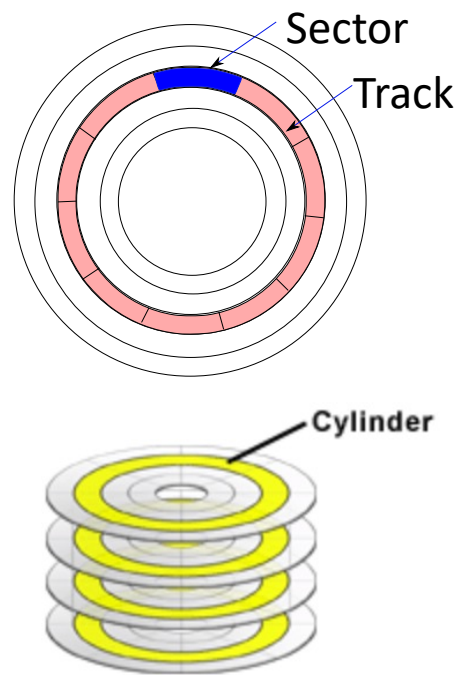
Cap 4: par. 4.4.1 – *"Dimensione dei blocchi"*

Dischi magnetici

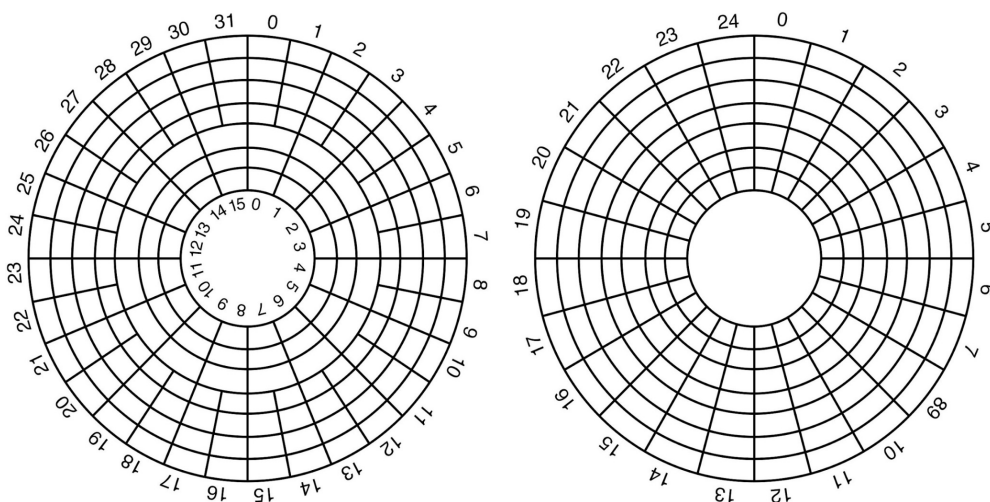


Dischi magnetici

- Un piatto è diviso in anelli concentrici (ciascun anello è detto traccia o **track**);
- Ciascuna traccia è divisa in settori (**sector**);
- Cilindro (**cylinder**): insieme delle tracce con la stessa distanza dal centro.

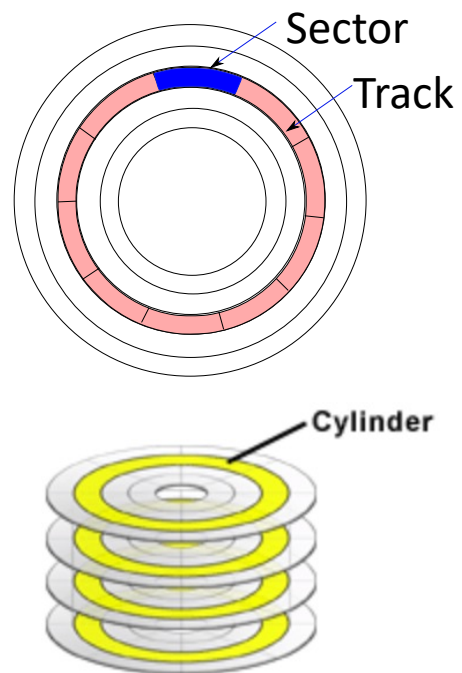


Geometria del disco



Indirizzamento

- ❑ **CHS**: cylinder-head-sector:
 - tripla (x, y, z);
- ❑ **LBA**: logical block addressing:
 - i settori sono numerati da 0, senza tener conto della geometria del disco.



Formattazione a basso livello

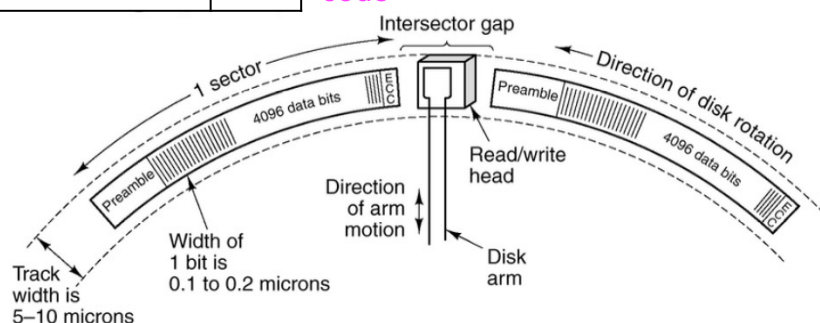
- ❑ **Tracce** concentriche, ciascuna contenente un certo numero di **settori** separati da "gap":
 - riduzione della capacità del disco di circa il 20%.

Preamble	Data	ECC
----------	------	-----

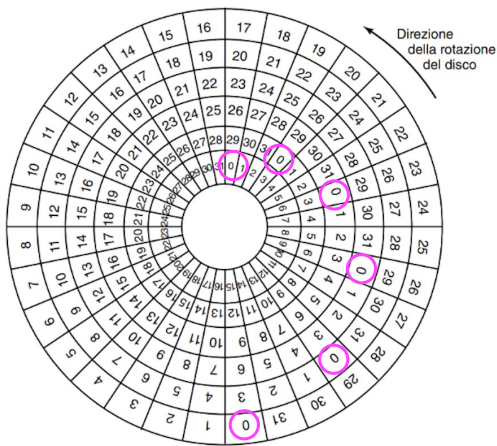
error correction code

“marcatore” indicante l’inizio di un settore

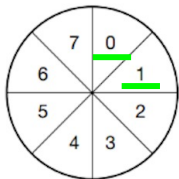
tipicamente i dischi usano settori di 512 byte



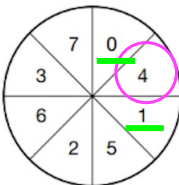
Formattazione a basso livello (ottimizzazioni)



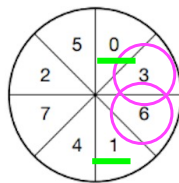
cylinder skew



no interleaving



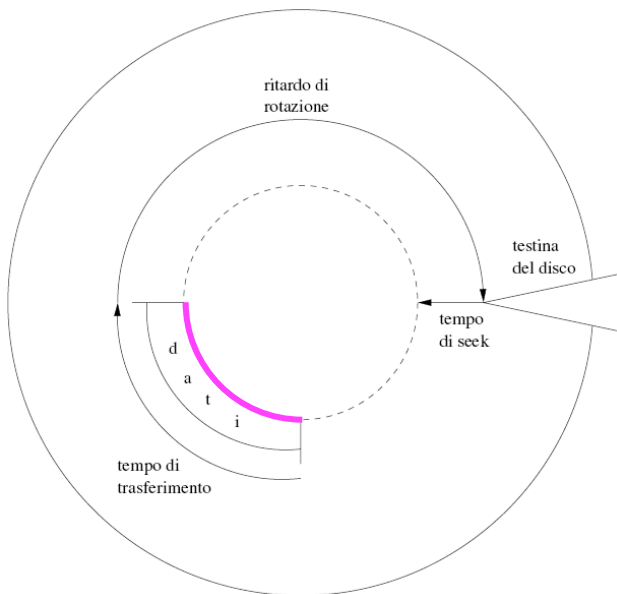
interleaving
singolo



interleaving
doppio

interleaving

Accesso al dispositivo



Tempo di seek (o tempo di *ricerca*): tempo medio per spostare la testina sul cilindro richiesto.

Ritardo di rotazione: tempo necessario affinché il disco effettui *mezza* rotazione (necessario a portare il settore sotto la testina).

Tempo di trasferimento: tempo necessario al trasferimento vero e proprio dei dati (tempo di rotazione *moltiplicato* la frazione di traccia da percorrere).

Accesso al dispositivo

- Esempio: tempo (in *ms*) per leggere un blocco di *k* **byte** in un disco con 1 MB (1048576 B) *per traccia*, tempo di ricerca medio pari a **5 ms** e tempo di rotazione pari a **8,33 ms**:

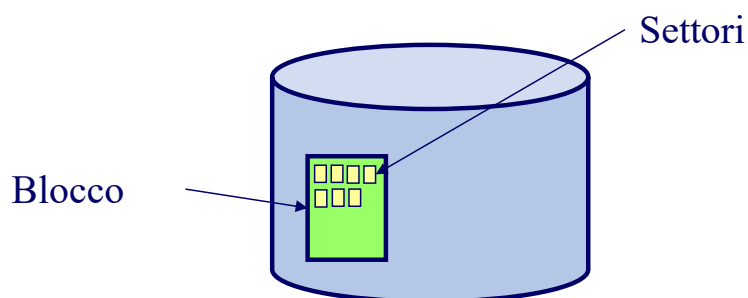
$$5 + 4,165 + (k / 1048576) * 8,33$$

$k = 1024B$	(1KB):	9.17ms
$k = 4096B$	(4KB):	9.20ms

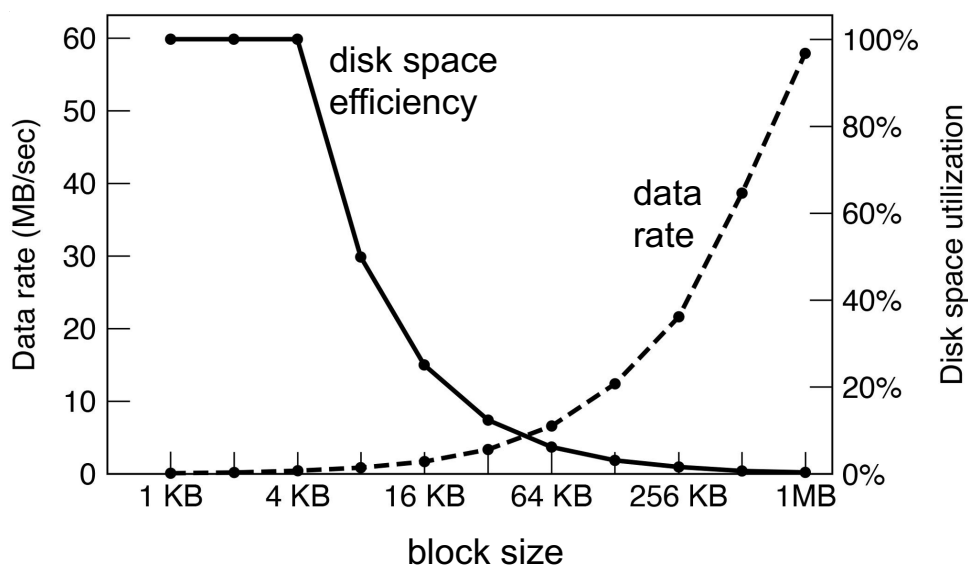
Dimensione dei blocchi

- Il sistema operativo usa una **dimensione del blocco** in generale **maggiore** di quella del **settore del disco**. Ogni blocco consiste di un certo numero di settori consecutivi:

la dimensione del blocco tipica è di **4KB**



Dimensione dei blocchi*



*Ipotesi: tutti i file hanno dimensione pari a 4K

FILE SYSTEM: IMPLEMENTAZIONE ED ESEMPI REALI

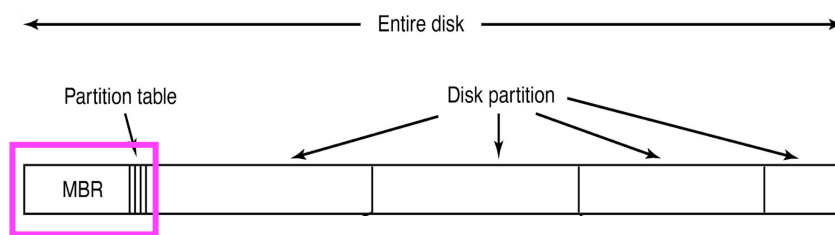
Cap 4: par. 4.3.1 – "*Layout del file system*"
 4.5.2 – "*File system UNIX*" (e nozioni introduttive
 "*File system ext2 di LINUX*" dal par.10.6.3)
 4.4.1 (paragrafo *Tenere traccia dei blocchi liberi*)
 4.3.4 – "*File condivisi*"

Layout del disco

- ❑ Un **disco** è suddiviso, *tipicamente*, in una o più **partizioni**, con file system indipendenti su ciascuna di esse (una partizione è vista come un disco *separato*);
- ❑ Il layout varia a seconda dei seguenti casi:
 - **MBR** (Master Boot Record – sistema con BIOS);
 - **UEFI** (Unified Extensible Firmware Interface).
- ❑ **Formattazione ad alto livello*** di ciascuna partizione:
 - predisposizione di un blocco di avvio (boot), del file system e del sistema operativo.

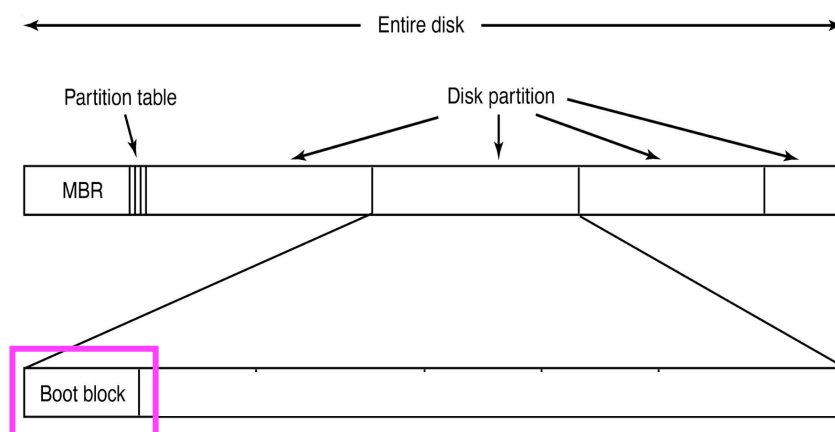
* la formattazione a **basso livello** consiste, invece, nella predisposizione di tracce e settori vista in precedenza.

Master Boot Record (MBR)



- **MBR**: settore 0 del disco – il BIOS legge ed esegue l'MBR all'avvio del computer;
- L'MBR contiene la **tabella delle partizioni** (fine MBR):
 - indirizzo-fine di ciascuna partizione;
 - indicazione della **partizione attiva**.

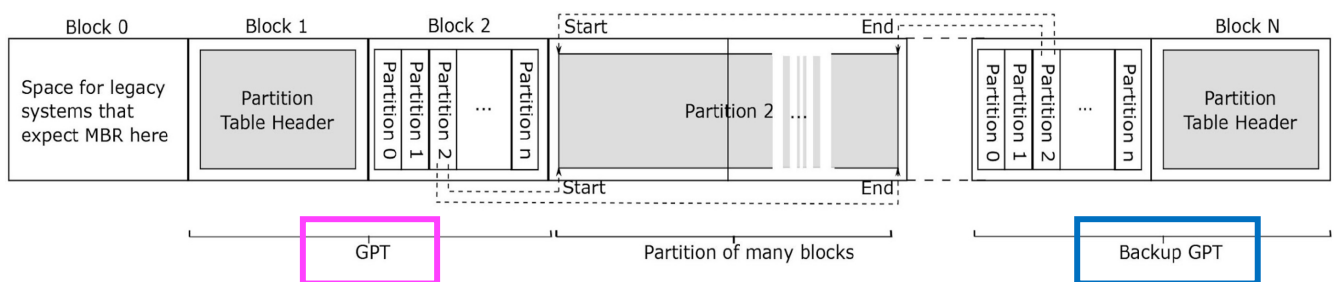
Master Boot Record (MBR)



- Segue lettura-esecuzione del **primo blocco** (**boot block**) della partizione attiva. Il programma nel *boot block* carica il SO contenuto in quella partizione.

Unified Extensible Firmware Interface (UEFI)

- ❑ La tabella delle partizioni è nel **secondo blocco**;
- ❑ La GPT – **G**UID (Globally Unique IDentifiers) **P**artition **T**able – contiene le informazioni sulle varie partizioni:
 - ❑ nel momento in cui viene trovata la GPT il firmware ha funzionalità sufficiente per leggere alcuni file system.



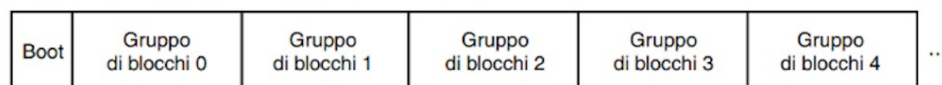
The Second Extended File System (ext2)

- ❑ Uno dei file system più noti nella community Linux (base delle distribuzioni attuali);
- ❑ Supporta tutte le caratteristiche del file system UNIX;
- ❑ Il file system è costituito da una serie di **blocchi** che possono essere letti o scritti.



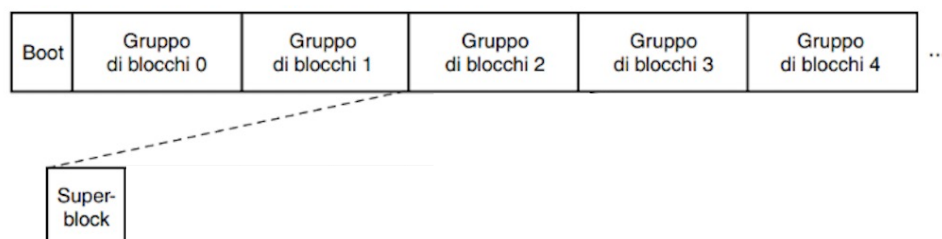
The Second Extended File System (ext2)

- ext2 è suddiviso in più parti, ossia i **gruppi di blocchi** (o **cylinder groups**) al fine di *localizzare* i file "nell'intorno" delle loro directory e *ridurre* i tempi di accesso (tale aspetto è collegato alla nozione di cilindro dei dischi).



The Second Extended File System (ext2)

- Superblock**: contiene le informazioni sul layout del file system (per es., numero di blocchi complessivo, blocchi / i-node liberi, etc.). Campi chiave:
 - block size**: *dimensione* del blocco (in byte);
 - first i-node**: numero del primo i-node del file system (il *primo inode* di un file system ext2 è l'entry per la directory /).

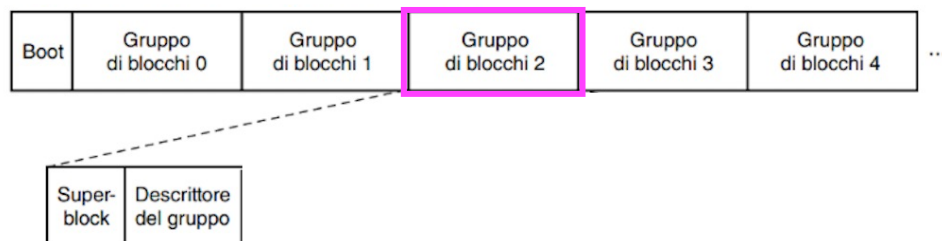


The Second Extended File System (ext2)

- ❑ **Descrittore del gruppo**: informazioni sul **gruppo** (per es., il numero di blocco della bitmap dei blocchi e degli i-node per il gruppo).

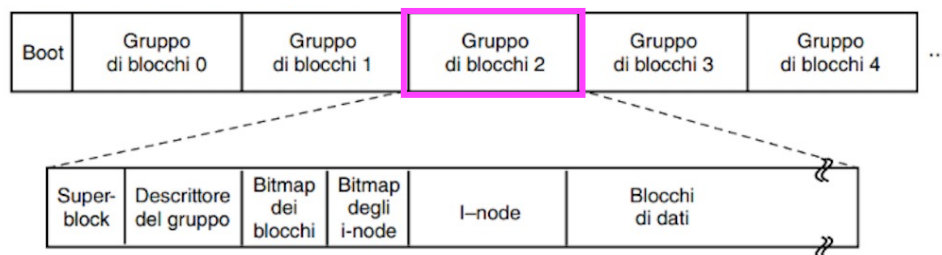
NOTA: In ogni gruppo, c'è una *copia* del superblock. I descrittori dei gruppi, sono accorpatis in una *tabella* e, ciascun gruppo, contiene una copia di tale tabella.

Solo le copie del gruppo 0 sono usate dal file system ext2 (le altre copie sono usate nel caso in cui la principale si corrompa).



The Second Extended File System (ext2)

- ❑ **Block bitmap**: mappa di bit (un bit per blocco di dati). Lo stato del bit indica se il relativo blocco è "libero" o "in uso";
- ❑ **i-node bitmap**: mappa di bit (un bit per i-node). Lo stato del bit indica se il corrispondente i-node è "libero" o "in uso";
- ❑ **i-node**: tabella degli i-node – ogni elemento corrisponde ad 1 file;
- ❑ **blocchi di dati (data blocks)**: blocchi del disco dove si trovano effettivamente i dati.



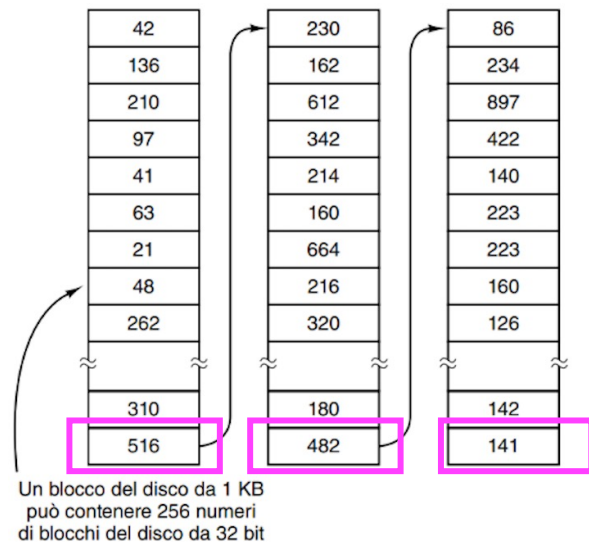
Sidenote: tracciare i blocchi "liberi"

❑ Soluzione 1: lista concatenata di blocchi:

- ciascun blocco contiene i numeri dei blocchi liberi (ad eccezione dell'ultimo).

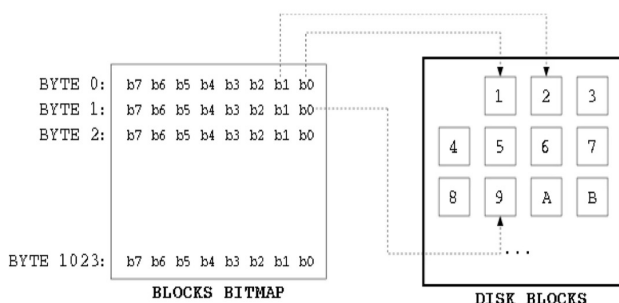
Esempio: se *dim. blocco* = 1KB (2^{10}) e il *numero del blocco* richiede 32 bit (4B) si potranno memorizzare $2^{10}/2^2 = 2^8$ (ossia 256*) numeri di blocco.

* a rigore 255, poichè l'ultimo numero di blocco è il numero del blocco successivo nella lista concatenata.



Sidenote: tracciare i blocchi "liberi"

❑ Soluzione 2: bitmap (mappa di bit): per ciascun bit, 1 → blocco libero / 0 → blocco usato.



1001101101101100
0110110111110111
1010110110110110
0110110110111011
1110111011101111
1101101010001111
0000111011010111
1011101101101111
1100100011101111
...
0111011101110111
1101111101110111

Esempio: se *dim. blocco* = 1KB si potrà memorizzare lo stato di $2^{10} \times 2^3 = 2^{13}$ (ossia 8192) blocchi.

Lookup di una directory

PREMESSA: come detto, l'i-node della directory radice ha una posizione nota sul disco.

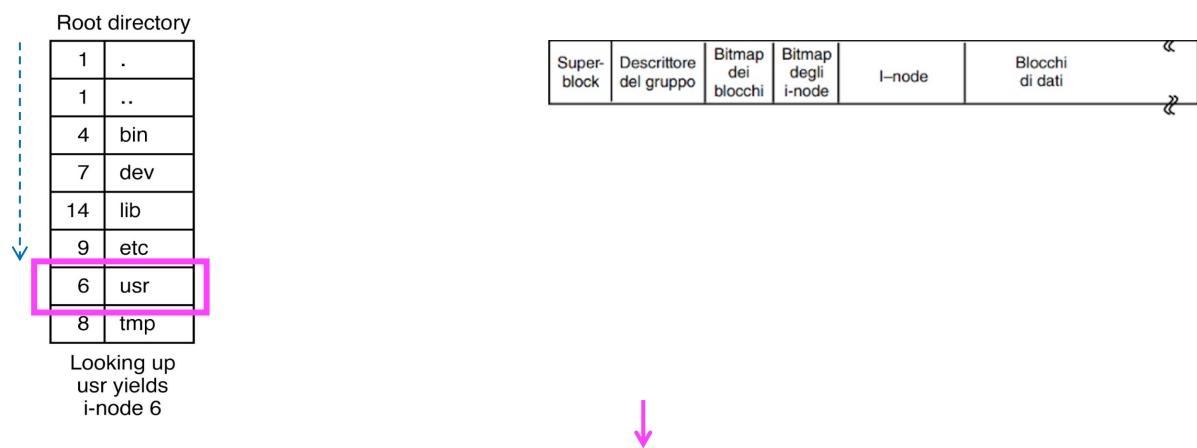


Figure 4-36. The steps in looking up **/usr/ast/mbox**.

Lookup di una directory

PREMESSA: come detto, l'i-node della directory radice ha una posizione nota sul disco.

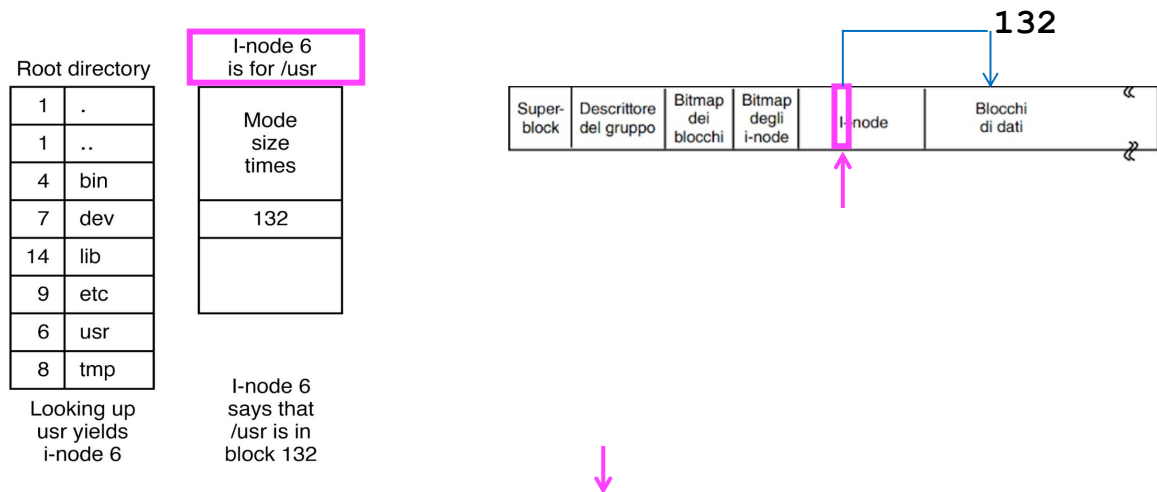


Figure 4-36. The steps in looking up **/usr/ast/mbox**.

Lookup di una directory

PREMESSA: come detto, l'i-node della directory radice ha una posizione nota sul disco.

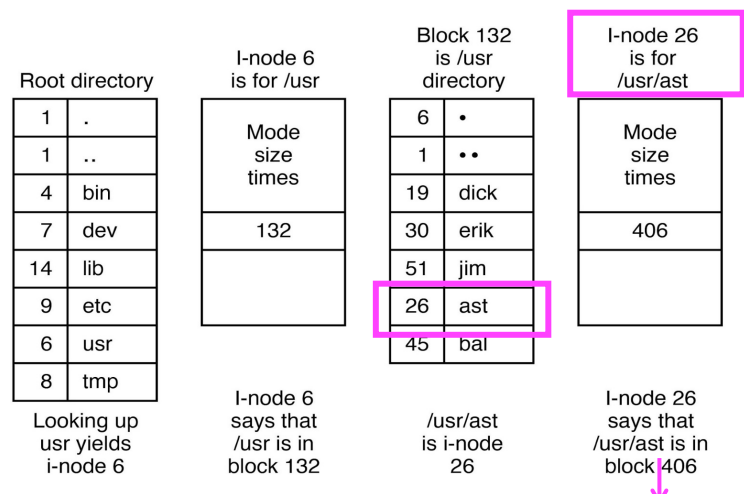


Figure 4-36. The steps in looking up `/usr/ast/mbox`.

Lookup di una directory

PREMESSA: come detto, l'i-node della directory radice ha una posizione nota sul disco.

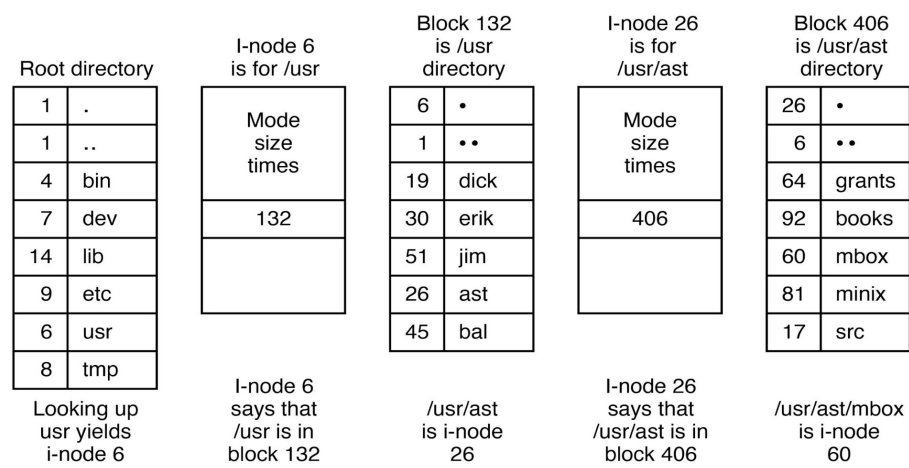


Figure 4-36. The steps in looking up `/usr/ast/mbox`.

File condivisi

Reminder: possibile struttura di una **directory entry**:

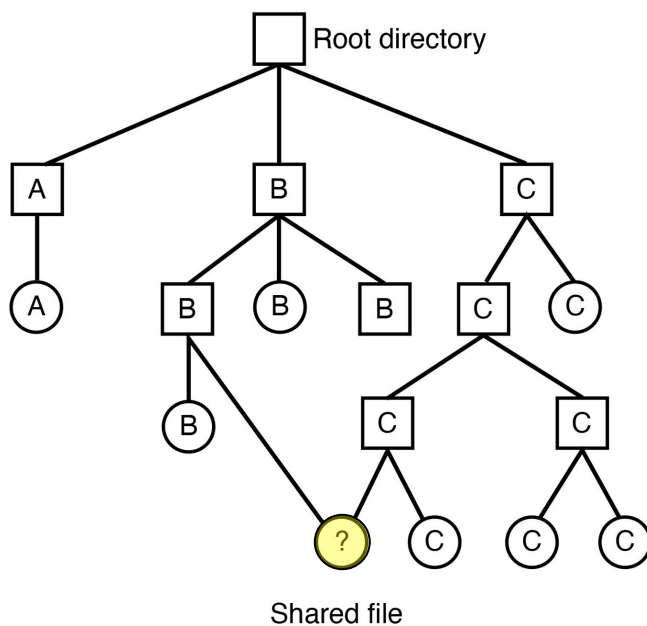
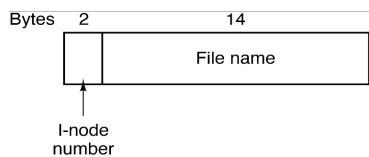
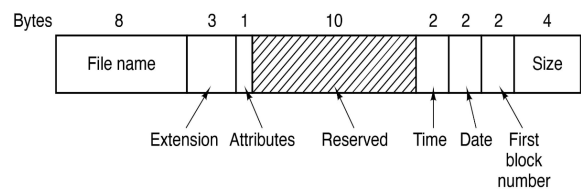


Figure 4-18. File system containing a shared file.

File condivisi (conteggio link nell'i-node)

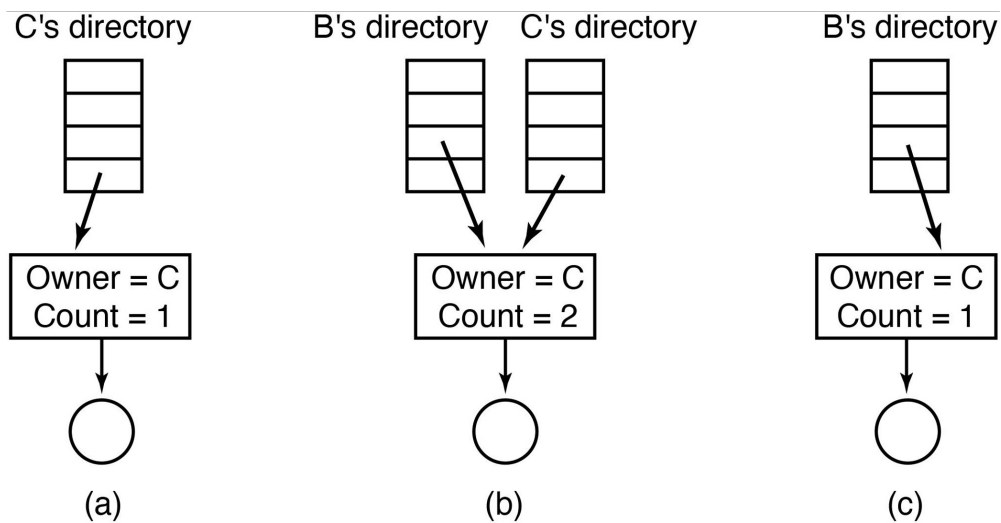
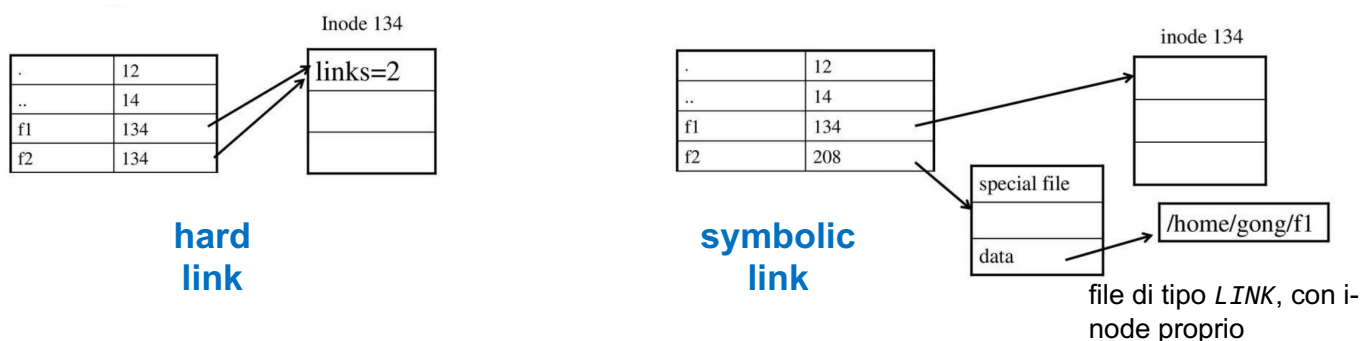


Figure 4-19. (a) Situation prior to linking. (b) After the link is created. (c) After the original owner removes the file.

File condivisi (hard link vs link simbolico)

- Creazione di un file di tipo **LINK**:
 - il file viene inserito nella directory di B;
 - il file *LINK* contiene solo il percorso del file a cui si collega.

Differenza tra i due casi



Creazione link: comando **ln**

- E' possibile creare **symbolic** (o *soft*) link oppure **hard** link con il comando **ln**:
 - la presenza dell'opzione **-s** indica che il link deve essere *soft*, altrimenti ne verrà creato uno *hard*.

```
$ ls -li mutex.c
925612 -rw-r--r-- 1 studente studente 2290 nov 13 15:01 mutex.c

$ ln -s mutex.c mutex_symlink.c // soft link
$ ls -li mutex_symlink.c
919650 lrwxrwxrwx 1 studente studente 7 nov 20 10:18 mutex_symlink.c -> mutex.c

$ ln mutex.c mutex_hardlink.c // hard link
$ ls -li mutex_hardlink.c
925612 -rw-r--r-- 2 studente studente 2290 nov 13 15:01 mutex_hardlink.c
```